

Machine Learning & Deep Learning: An Introduction

A Beginner-Friendly Tour of the Algorithms Behind Modern AI — Session 08/25/2026

01 / FOUNDATIONS

Core Concepts

- **What is ML, really?**
From hand-written rules to learned rules
- **Linear Regression**
Predicting continuous numbers
- **Regularization**
Keeping models simple (Ridge & Lasso)
- **Logistic Regression**
Predicting yes/no probabilities

02 / CLASSIC ALGORITHMS

Classification & Ensembles

- **Neighbors, Margins & Trees**
k-NN, SVM, and Decision Trees
- **Teamwork Models**
Random Forests & Boosting
- **Bonus Analytics Tools**
Forecasting, cause vs. coincidence, and catching outliers

03 / DEEP LEARNING & BEYOND

Neural Networks & Success

- **How Networks Learn**
Layers, weights, and gradient descent
- **Attention & Transformers**
The revolutionary idea behind ChatGPT
- **Model Evaluation**
How do we know if a model is actually any good?

What Is Machine Learning, Really?

01 / PARADIGM SHIFT

Traditional vs. ML

Traditional Programming

You write the rules, feed in the data, and get answers.

Machine Learning Flips It

You feed in data + known answers, and the computer works out the rules.

Example: *Instead of hand-coding complex loan approval rules, show the model thousands of past outcomes and let it detect patterns.*

02 / CORE COMPONENTS

Every Model Has 3 Parts

1. Representation

How the model turns inputs into a prediction (a line, a decision tree, or a neural network).

2. Evaluation

A loss function that scores how far off or "wrong" a prediction was from reality.

3. Optimization

An algorithm to systematically adjust the model parameters to make that loss smaller.

03 / EVOLUTION

The Maturity Ladder

Descriptive

What happened? (dashboards, simple aggregations)

Diagnostic

Why did it happen? (root-cause, correlation studies)

Predictive **ML MAIN ZONE**

What will happen next? This is where most modern ML algorithms live.

Prescriptive

What should we do? (decision engines, optimization)

Linear Regression

$$\theta = (X^T X)^{-1} X^T y$$

The formula that finds the best-fit line directly

What Linear Regression Does

- **Best-Fit Line:** Draws the straight line that best fits a scatter of points.
- **Least Squares:** 'Best fit' means the line that minimizes total squared distance to every point.
- **One-Step Solution:** Can be solved exactly in one step using the Normal Equation formula.
- **Linearity:** Works best when the relationship really is roughly a straight line.

$$\text{Error} = \text{average of } (\text{actual} - \text{predicted})^2$$

How we measure how 'wrong' a line is

Gradient Descent (Alternative Fit)

- **The Analogy:** Like a hiker lost in thick fog, feeling for the steepest downhill slope underfoot at each step.
- **Scalability:** Useful when the exact formula is too slow (huge datasets, many features).
- **Learning Rate:** The step size controls how big each downhill step is.
- **Step Tuning:** Too big a step overshoots; too small takes forever to reach the bottom.

PREDICTING YES/NO (OR ONE OF SEVERAL CLASSES)

Logistic Regression

$$\sigma(z) = 1 / (1 + e^{-z})$$

The Sigmoid — squashes any number into a 0–1 probability

How It's Trained & Used

- **Probability Output:** Maps inputs to a continuous value between 0 and 1, representing class probability.
- **Loss Function:** Uses cross-entropy loss to measure how far predictions are from the true 0 or 1 labels.
- **Optimization:** Adjusts model weights iteratively to maximize correct class confidence and shrink total loss.

softmax* → *picks probabilities

Same core concept, extended to multi-class scenarios

Reading the Model

- **Classification:** Despite its name, it is fundamentally a classification method, not a regression one.
- **Decision Boundary:** Draws a straight decision boundary that separates classes as cleanly as possible.
- **Coefficients:** Each coefficient indicates how strongly a specific feature pushes the output toward 'yes' or 'no'.
- **Multi-Class:** Extends naturally to predict multiple categories (e.g., cat, dog, bird) using a softmax layer.

Keeping Models Simple: Regularization

01 / L2 Regularization

Ridge

Error + a penalty for large coefficients

Shrinks every coefficient a little

In Plain Words:

Discourages any coefficient from getting too big

Effect:

Shrinks all coefficients, none reach zero

Best Use Case:

Many features that overlap/correlate

02 / L1 Regularization

Lasso

Error + a penalty for non-zero coefficients

Can shrink some coefficients to zero

In Plain Words:

Also discourages using unnecessary features at all

Effect:

Can drive weak coefficients to exactly zero

Best Use Case:

Automatically drop useless features

03 / Combined Penalty

ElasticNet

A blend of both ideas at once

Combines L1 and L2 penalties

In Plain Words:

A blend of both Ridge and Lasso ideas

Effect:

Shrinks and can zero out coefficients

Best Use Case:

Lots of correlated features, want some sparsity

"YOU ARE THE AVERAGE OF YOUR NEIGHBORS"

k-Nearest Neighbors (k-NN)

How It Works

- To predict a new point, find its k closest points in the training data
- Classification: let those neighbors vote on the label
- Regression: average their values
- There's no real 'training' — it just remembers all the data

Choosing k

- Small k (like 1): follows the data very closely — can be noisy
- Large k: smooths things out — can miss real patterns
- Pick k by trying a few values and checking what predicts best

Distance Used to Find Neighbors	Idea
Euclidean	Straight-line distance (as the crow flies)
Manhattan	Distance moving only along grid lines (city blocks)
Minkowski	A general formula that includes both above

Where It Struggles

- Slow to predict on huge datasets — checks distance to every point
- Features on different scales (e.g. age vs. income) can distort distance
- With too many features, everything starts looking equally 'far away'

VISUALIZING K-NEAREST NEIGHBORS (k-NN)

STEP 1: CALCULATE DISTANCES

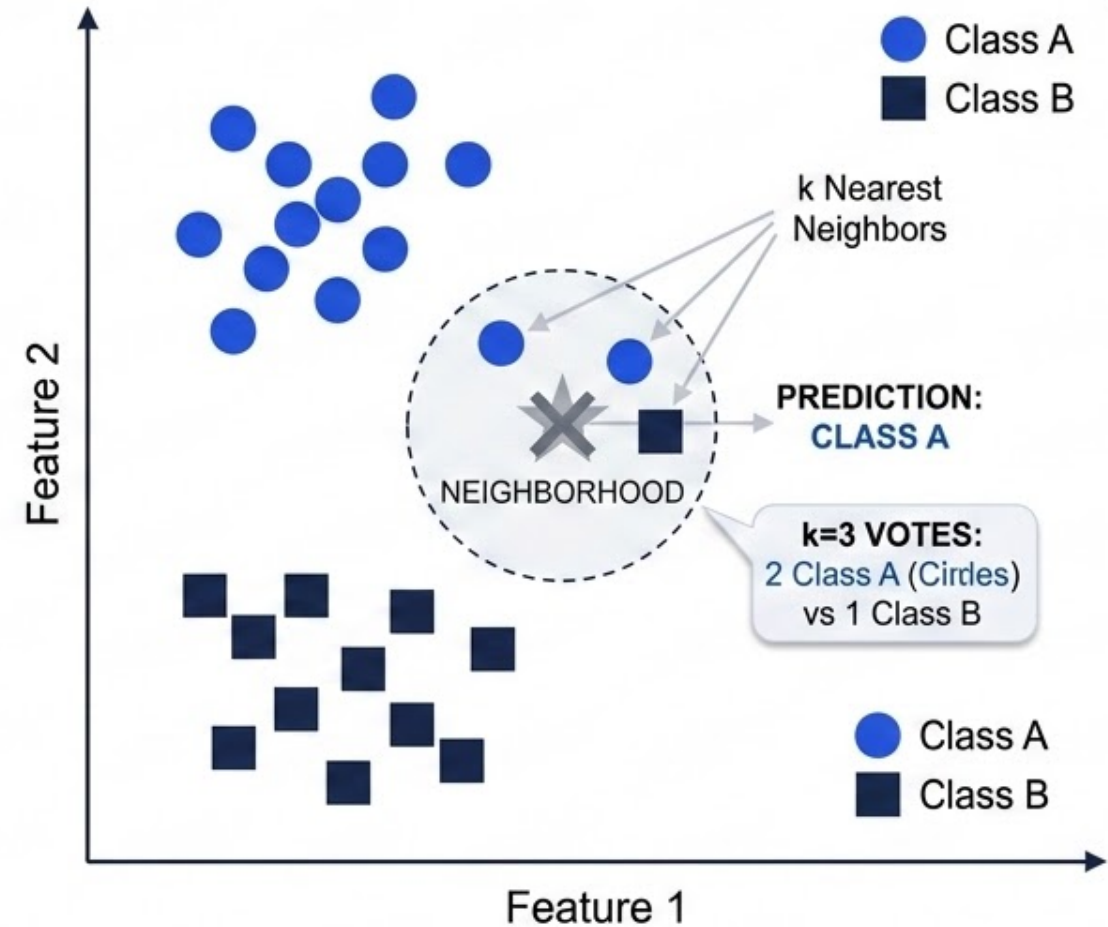
- Measure distance from the new point to all training data points.
- (e.g., Euclidean Distance, Manhattan Distance)

STEP 2: FIND K CLOSEST

- Identify the specific 'k' data points with the smallest calculated distances.
- (e.g., k=3 is chosen here)

STEP 3: POLL VOTES / AVERAGE

- For classification, count the labels of the k neighbors and assign the majority class.
- (Regression alternative: Average the numerical values)



DRAW THE WIDEST POSSIBLE STREET BETWEEN CLASSES

Support Vector Machines (SVM)

Find the boundary line that maximizes the gap to the nearest points on each side

The 'margin' — SVM's core idea

tune Margins & Support Vectors

- **Support vectors:** The handful of closest points that actually define the boundary line.
- **Free movement:** Every other data point could move around freely without changing the decision boundary at all.
- **C as a dial:** Low C allows some margin violations for a wider, more tolerant boundary "street".
- **High C constraint:** Insists on zero mistakes, forcing a narrower, much stricter boundary.

blur_on The Kernel Trick

- **Non-linear limits:** Often classes can't be split by a straight line—such as two concentric, nested circles.
- **Curved boundaries:** Kernels allow SVM to map data into higher dimensions to draw complex curves without explicit coordinate math.
- **Linear kernel:** Standard straight-line division for linearly separable scenarios.
- **RBF kernel:** Highly flexible, curved boundary representing a highly reliable default choice.

DRAW THE WIDEST POSSIBLE STREET BETWEEN CLASSES

Support Vector Machines (SVM)

Find the boundary line that maximizes the gap to the nearest points on each side

The 'margin' — SVM's core idea

tune Margins & Support Vectors

- **Support vectors:** The handful of closest points that actually define the boundary line.
- **Free movement:** Every other data point could move around freely without changing the decision boundary at all.
- **C as a dial:** Low C allows some margin violations for a wider, more tolerant boundary "street".
- **High C constraint:** Insists on zero mistakes, forcing a narrower, much stricter boundary.

blur_on The Kernel Trick

- **Non-linear limits:** Often classes can't be split by a straight line—such as two concentric, nested circles.
- **Curved boundaries:** Kernels allow SVM to map data into higher dimensions to draw complex curves without explicit coordinate math.
- **Linear kernel:** Standard straight-line division for linearly separable scenarios.
- **RBF kernel:** Highly flexible, curved boundary representing a highly reliable default choice.

Decision Trees

help_outline

The Basic Idea

- Repeatedly ask yes/no questions about features (e.g., "*Is age > 30?*") to split the data.
- Each split tries to make the resulting groups as '**pure**' as possible (mostly one class).
- Keep splitting until groups are pure enough or a stopping rule kicks in.

insights

How We Score a Split

Gini Impurity

How often you'd mislabel a random point if you guessed randomly based on the group's class distribution.

Entropy / Info Gain

How much 'surprise' or disorder is reduced in the group after performing the split.

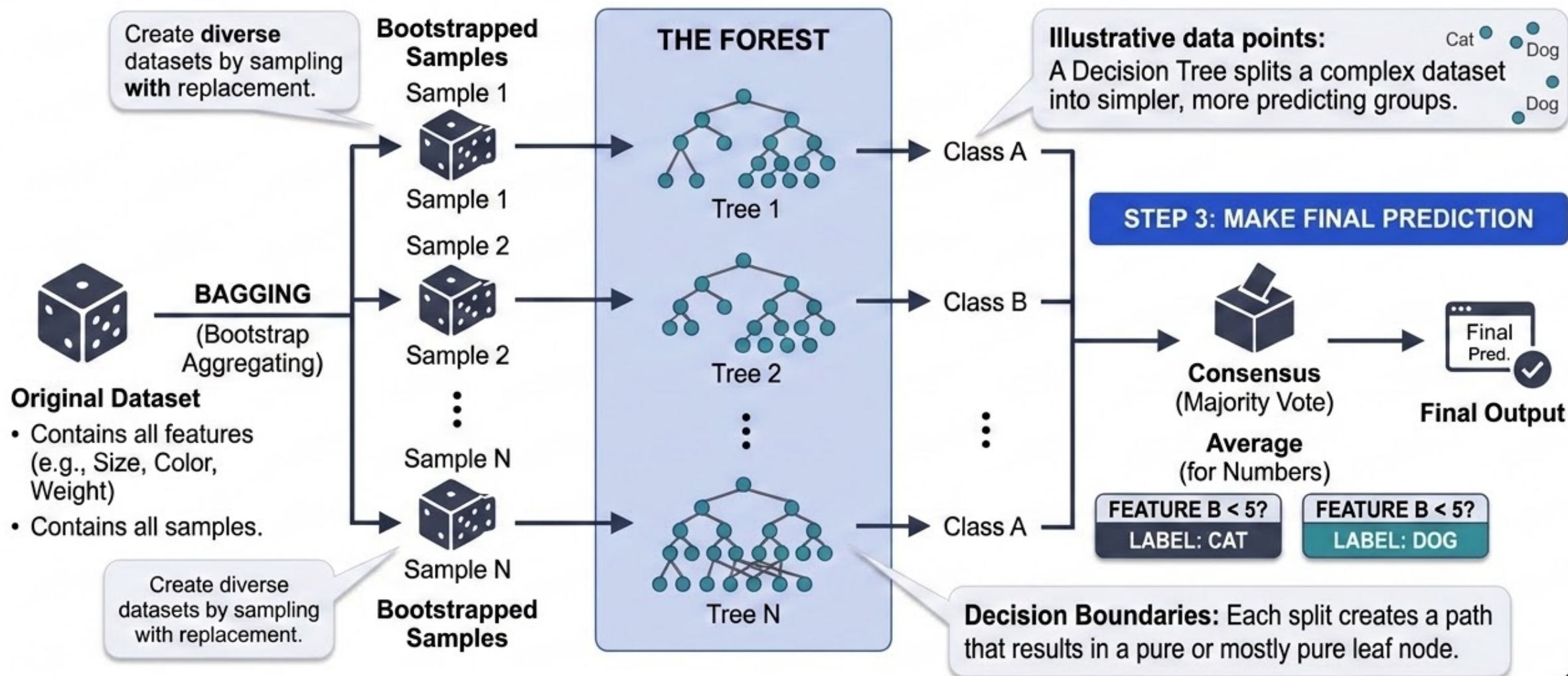
report_problem

The Overfitting Trap

- A tree grown all the way down can perfectly memorize the training data — and generalize badly.

Pruning (limiting depth, requiring a minimum group size) keeps the tree robust and prevents memorization.

DECISION TREES: How They Work (A Simple Walkthrough)



Random Forests

The Core Idea

- A single decision tree is easily fooled by noise in its training data
- So instead, train hundreds of trees, each on a random, slightly different sample of the data
- Let every tree vote, and go with the majority (or average, for numbers)
- One tree's mistakes tend to get outvoted by the rest

One Extra Trick

- Each tree is also only shown a random subset of the features at every split
- This keeps the trees from all making the exact same mistakes
- More variety among the trees makes the group vote more reliable

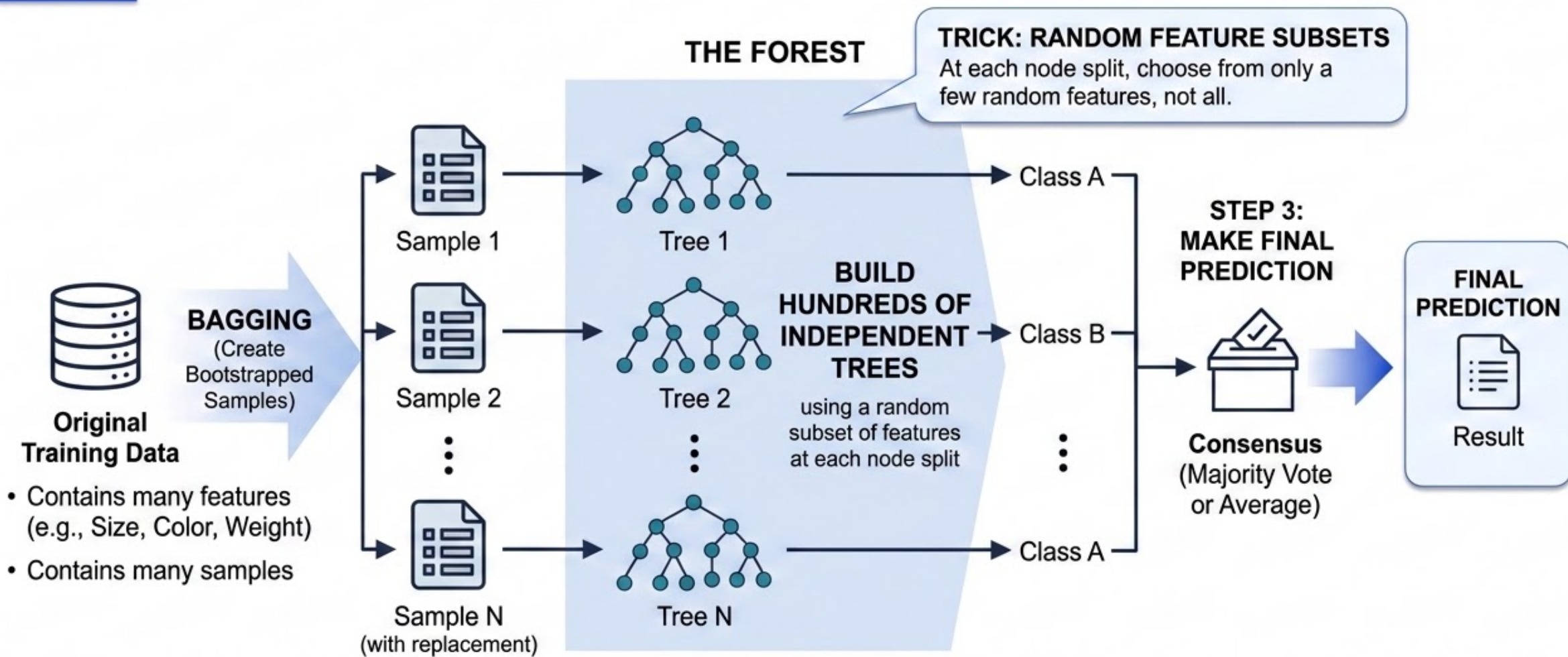
A Built-In Report Card

- Since each tree only sees part of the data, the leftover part can quietly test it
- This gives a free, ready-made estimate of how well the forest generalizes
- No separate test set required to get this estimate

Which Features Mattered?

- Random Forests can rank features by how much they helped reduce mistakes
- Useful for understanding what's actually driving the predictions

UNDERSTANDING THE RANDOM FOREST PROCESS



EACH NEW TREE FIXES THE LAST ONE'S MISTAKES

Boosting & XGBoost

New prediction = old prediction + (a small tree trained on the leftover errors)

The boosting loop, repeated many times

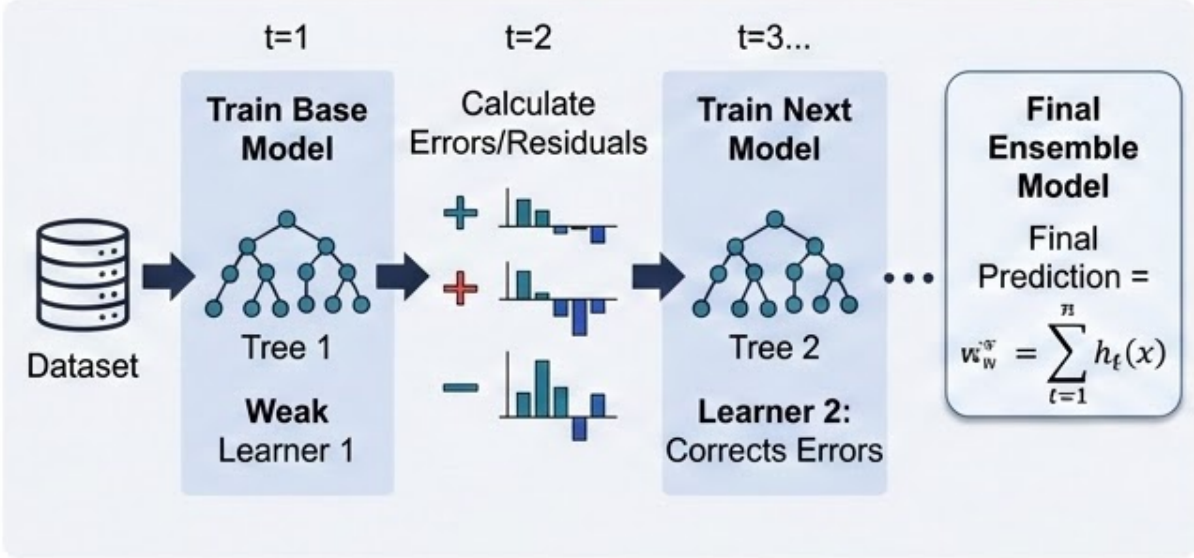
	Random Forest (Bagging)	XGBoost (Boosting)
How trees are built	All at once, independently	One at a time, each learning from the last one's errors
Good analogy	A panel of independent experts voting	A student redoing a draft based on feedback

Why XGBoost Is So Popular

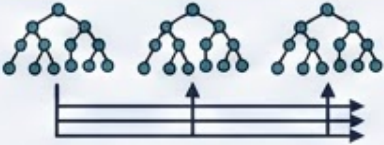
- Usually squeezes out very high accuracy on structured/tabular data
- Has built-in safeguards against overfitting (it penalizes overly complex trees)
- A 'learning rate' controls how much each new tree is allowed to change the prediction

BOOSTING & XGBOOST: AN ILLUSTRATED GUIDE

HOW BOOSTING WORKS (SEQUENTIAL LEARNING)



BAGGING VS. BOOSTING: AN ILLUSTRATED COMPARISON

	Random Forest (Bagging)	XGBoost (Boosting)
Building Trees	 Built simultaneously	 Built Interdependency
Good Analogy	 Wisdom of the Crowd	 Iterative Feedback Loop

KEY XGBOOST FEATURES (VISUALIZED)



Highly Accurate:
Learns complex patterns by minimizing residual error.



Regularization Shield:
Built-in safeguards keep trees simple and prevent overfitting.



Learning Rate Control:
Determines how quickly or cautiously each tree updates the model.



Three More Useful Tools



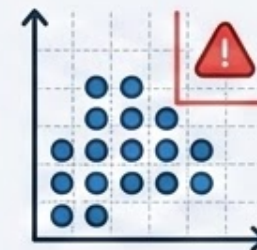
Causal Inference

- Just because two things move together doesn't mean one causes the other
- These methods try to isolate the actual effect of one change on an outcome
- Answers 'what would happen if we did X?' — not just 'what happened when X occurred?'
- Important for decisions, not just predictions



Time-Series Forecasting

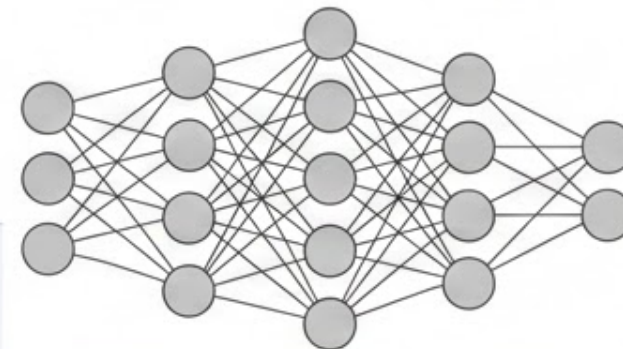
- Predicting future values from a sequence of past values (sales, traffic, temperature...)
- Look for a trend (long-term direction) and seasonality (repeating patterns)
- Always test by predicting the future from the past — never shuffle time-ordered data



Anomaly Detection

- Finding the unusual data points — fraud, defects, sensor glitches
- Isolation Forest: odd points are easier to 'isolate' with just a few random splits
- Works without needing any labeled examples of what an anomaly looks like

How Neural Networks Learn



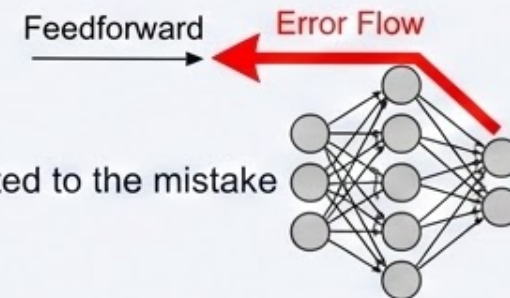
The Structure

- A neural network is layers of simple units ('neurons') stacked one after another
- Each neuron takes a weighted sum of its inputs, then passes it through a small non-linear function so the network can learn curves, not just straight lines

Activation Function	What It Does
ReLU	Passes positive values through, zeroes out negative ones — simple and fast
GELU	A smoother version of ReLU, common in today's large models

How Training Works: Backpropagation

- Make a prediction, compare it to the true answer, measure the error
- Backpropagation traces that error backward through the network to find out how much each weight contributed to the mistake
- The optimizer (e.g. Adam) then nudges every weight slightly to reduce that error, and repeat



Attention & Transformers

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Formal version — every word looks at every other word and decides how relevant it is

In Plain Words

- Every word in a sentence 'looks around' at all the other words
- It decides which other words are most relevant to understanding it right now
- Example: in "the animal didn't cross the street because it was tired", attention helps the model figure out 'it' means the animal, not the street
- The model then blends information from the relevant words to build understanding

Why This Was a Big Deal

- Older models read text one word at a time, in order — slow, and easy to 'forget' earlier words
- Attention lets a model look at the whole sentence at once, in parallel — much faster to train
- This is the core building block inside GPT, BERT, and most modern language models

Is the Model Actually Good?

Too Simple vs. Too Complicated

- Underfitting (high bias): e.g. fitting a straight line to data that actually curves like a parabola
- Overfitting (high variance): the model memorized noise instead of the real pattern
- The goal is the sweet spot in between — check this by testing on data the model never saw

Metric	The Question It Answers
Precision	Of everything I flagged as 'yes', how much was right?
Recall	Of everything that was actually 'yes', how much did I catch?
F1	A single score balancing precision and recall

Picking the Right Metric

- Example: spam filter — missing a spam email (low recall) is annoying but a false alarm blocking a real email (low precision) may be worse
- There's no universally 'best' metric — pick the one that matches what mistakes actually cost your business
- Accuracy alone can be misleading when one class is much rarer than the other

WHICH TOOL DO I REACH FOR?

Putting It All Together

If your problem looks like...	A good starting point is...
Small/medium data, need to explain the result	Linear/Logistic Regression, a single Decision Tree
Tabular data, want the best accuracy	Random Forest or XGBoost
Small dataset with a clear separation between groups	SVM
Quick baseline, small dataset	k-NN
Images or grid-like data	Deep learning (CNNs)
Text or sequences	Transformers (attention-based)
'What happens if we change X?' questions	Causal inference
Finding rare/unusual events	Isolation Forest (anomaly detection)